

Databricks Architecture, Governance Models & Platform Capabilities



A practitioner's reference on the Lakehouse Platform: control-plane / data-plane architecture, Unity Catalog governance, compute, storage, orchestration, and AI capabilities — with worked examples for engineers, architects, and data governance teams.

DOCUMENT TYPE	Technical Reference & Study Guide
AUDIENCE	Data Engineers, Platform Architects, Data Governance & Security Teams
VERSION	1.0
CREATED	July 26, 2026, 20:21 PDT
PREPARED FOR	Internal Knowledge Base / RAG Retrieval Corpus

Table of Contents

1	Executive Summary	4
2	The Databricks Lakehouse Platform: Overview	5
2.1	From Data Warehouse to Lakehouse	5
2.2	The Four Platform Pillars	5
3	Reference Architecture: Control Plane and Data Plane	6
3.1	The Control Plane	6
3.2	The Data Plane and the Hybrid Deployment Model	6
3.3	The Serverless Data Plane	6
4	Delta Lake: The Open Storage Foundation	7
4.1	ACID Transactions and the Transaction Log	7
4.2	The Medallion Architecture	7
4.3	Performance: Liquid Clustering, Z-Ordering, and Predictive I/O	8
5	Compute Architecture: Clusters, SQL Warehouses, and Photon	9
5.1	All-Purpose and Job Clusters	9
5.2	SQL Warehouses: Serverless, Pro, and Classic	9
5.3	The Photon Engine	9
6	Unity Catalog: Unified Governance for Data and AI	11
6.1	The Three-Level Namespace and Metastore	11
6.2	Governance Object Model	11
6.3	2026 Additions: Business Glossary, Domains, and Governance Hub	12
7	Data Governance Models: Access Control, Lineage, and Compliance	13

7.1	RBAC and ABAC Access Models	13
7.2	Row, Column, and Dynamic View Security	13
7.3	Lineage, Audit, and Classification	14
8	Data Sharing and Interoperability	15
8.1	Delta Sharing	15
8.2	Lakehouse Federation	15
9	Orchestration and Pipelines: Lakeflow and Workflows	16
9.1	Databricks Workflows and Jobs	16
9.2	Lakeflow: Connect, Declarative Pipelines, and Jobs	16
10	AI and Machine Learning: MLflow, Mosaic AI, and Genie	17
10.1	MLflow and the Model Lifecycle	17
10.2	Mosaic AI: Governed Generative AI	17
10.3	Genie: Conversational Analytics	18
11	Security, Compliance, and Enterprise Readiness	19
12	Best Practices, Roadmap & Conclusion	20
12.1	Conclusion	20
13	References	21

SECTION 1

Executive Summary

Why the Lakehouse pattern, and this document, matter to any team running data and AI workloads at scale.

Databricks is a unified data and AI platform built around the **Lakehouse** pattern: a single system that combines the low-cost, flexible storage of a data lake with the transactional reliability, performance, and governance of a data warehouse. Rather than maintaining separate systems for raw storage, structured analytics, business intelligence, and machine learning, organizations run all of these workloads against one governed copy of the data.

This document is written as a practitioner's reference. It covers four areas that any Databricks professional — whether a data engineer, platform architect, ML engineer, or governance lead — needs to reason about clearly:

- **Architecture.** How Databricks separates the control plane (management, orchestration, UI) from the data plane (compute and storage that hold customer data), and how this shapes network design, security boundaries, and cost.
- **Storage and compute.** Delta Lake as the transactional storage format underneath every table, and the compute options — clusters, SQL warehouses, serverless, and the Photon engine — used to process it.
- **Governance.** Unity Catalog as the unified governance layer for tables, files, models, notebooks, and — increasingly — AI agents, along with the access control models (RBAC/ABAC), lineage, and audit capabilities that make regulated workloads possible.
- **Platform capabilities.** Orchestration (Workflows, Lakeflow), data sharing (Delta Sharing, Lakehouse Federation), and the AI stack (MLflow, Mosaic AI, Genie) that sit on top of the governed data layer.

HOW TO USE THIS DOCUMENT

Each section stands on its own and includes a diagram, a plain-language explanation, and a worked example (Python or SQL) so the concept can be verified against real syntax rather than taken on faith. Section 12 consolidates the material into an adoption checklist and conclusion, and Section 13 lists the primary sources used to compile this reference.

The throughline across every section is that Databricks treats **architecture and governance as the same problem**: because compute is ephemeral and storage is the durable asset, governance controls are enforced at the storage and metadata layer (Unity Catalog) rather than bolted onto each individual compute engine. This is what allows the same permission model to cover a SQL query, a Python notebook, an ML model, and — as of 2026 — an autonomous AI agent.

SECTION 2

The Databricks Lakehouse Platform: Overview

What the Lakehouse pattern is, why it emerged, and the four pillars the platform is organized around.

2.1 From Data Warehouse to Lakehouse

For roughly two decades, enterprise data platforms split into two lineages. **Data warehouses** (Teradata, Oracle, later Snowflake and BigQuery) gave clean SQL semantics, strong governance, and fast BI queries, but stored data in proprietary formats that were expensive to scale and poorly suited to unstructured data, video, or ML training sets. **Data lakes** (Hadoop HDFS, then cloud object storage such as S3, ADLS, and GCS) gave cheap, scalable, open storage for any file type, but lacked transactional guarantees — concurrent writers could corrupt tables, and there was no built-in schema enforcement or governance.

Most organizations ended up running both: a lake for raw/ML data and a warehouse for curated BI data, connected by fragile ETL pipelines that copied data between them, doubling storage cost and creating two sources of truth that drifted apart. The **Lakehouse** architecture, introduced by Databricks and formalized in a 2021 CIDR paper, closes this gap by adding a transactional metadata layer (Delta Lake) directly on top of open object storage, so a single copy of the data can serve BI, streaming, data science, and AI workloads with warehouse-grade reliability.

2.2 The Four Platform Pillars

Everything in the rest of this document maps onto four pillars that make up the platform:

Pillar	Core Technology	What It Solves
Storage	Delta Lake (open table format on cloud object storage)	ACID transactions, schema enforcement, time travel, and a single copy of data for every workload
Compute	Clusters, SQL Warehouses, Photon, serverless compute	Elastic, workload-specific processing that scales independently of storage
Governance	Unity Catalog	One permission, lineage, and audit model across clouds and asset types (tables, files, models, agents)
Workflows & AI	Lakeflow, Workflows, MLflow, Mosaic AI, Genie	Orchestration, ML lifecycle management, and generative AI built directly on governed data

WHY THIS MATTERS FOR A RAG OR KNOWLEDGE-BASE PROJECT

Databricks documentation is unusually layered: a term like "cluster" or "catalog" means something specific and load-bearing. A retrieval system built over this material benefits from chunking that respects these section boundaries, since a question like "how does Unity Catalog handle row-level security" should retrieve the governance-model section (7.2), not the general overview (2.2).

SECTION 3

Reference Architecture: Control Plane and Data Plane

The hybrid deployment model that defines every security and networking decision on the platform.

Databricks runs as a **hybrid architecture** split into two planes. This single design decision is the reason Databricks can offer a managed SaaS experience while a customer's raw data never has to leave their own cloud account.

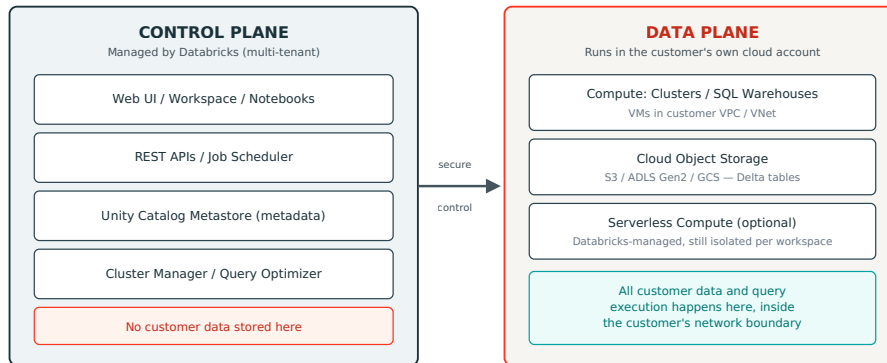


Figure 3.1 — Control plane / data plane split. The control plane (multi-tenant, Databricks-managed) coordinates jobs and stores metadata; the data plane (single-tenant, customer cloud account) executes compute against the customer's own storage.

3.1 The Control Plane

The control plane is multi-tenant infrastructure that Databricks operates and manages. It hosts the web application (notebooks, workspace browser, dashboards), REST and SQL APIs, the job scheduler, cluster manager, and the Unity Catalog metastore that tracks metadata (table names, schemas, permissions, lineage) — but not the underlying data itself. Because the control plane never stores customer data at rest, a security review of the control plane is largely a review of Databricks' own SaaS operational controls rather than the customer's data footprint.

3.2 The Data Plane and the Hybrid Deployment Model

The classic data plane is **single-tenant**: compute resources (VMs backing clusters and classic SQL warehouses) run inside the customer's own cloud account (AWS, Azure, or GCP), inside a VPC/VNet that the customer controls. Databricks orchestrates what runs, but the compute itself, the network egress rules, and the storage all sit under the customer's cloud billing and IAM boundary. This is sometimes called a "hybrid PaaS" model: control is centralized and managed; data and execution are decentralized and customer-owned.

3.3 The Serverless Data Plane

Serverless compute (serverless SQL warehouses, serverless jobs, serverless notebooks, and serverless GPU compute for model serving) moves the data plane into a Databricks-managed serverless layer, still logically isolated per workspace and encrypted, but without the customer needing to provision or manage the underlying VMs. This trades a small amount of infrastructure control for near-instant start times and fine-grained, usage-based billing — the serverless SQL warehouse typically starts in seconds versus minutes for a classic warehouse.

WORKED EXAMPLE — WHICH PLANE DOES THIS TOUCH?

A data engineer opens a notebook (control plane UI), attaches it to a job cluster that spins up in their AWS account (data plane), reads a Delta table from S3 (data plane storage), and the query plan is optimized by the Databricks control plane before execution happens entirely in the data plane. Only metadata about the query — not the underlying rows — ever crosses back into the control plane.

SECTION 4

Delta Lake: The Open Storage Foundation

The open table format that gives warehouse-grade reliability to files sitting in cloud object storage.

Delta Lake is an open-source storage layer that sits on top of Parquet files in cloud object storage and adds a transaction log, schema enforcement, and versioning. It is the format underneath virtually every managed table in Databricks and is also readable by Spark, Trino, Presto, and — via UniForm — by Iceberg and Hudi clients.

4.1 ACID Transactions and the Transaction Log

Every write to a Delta table (insert, update, delete, merge) is recorded as an ordered, atomic entry in the `_delta_log` directory — a set of JSON commit files plus periodic Parquet checkpoints. Readers use this log to construct a consistent snapshot of the table, which is what makes concurrent readers and writers safe without external locking, and what enables **time travel** (querying the table as of a previous version or timestamp).

`/warehouse/sales_orders/` (Delta table on object storage)

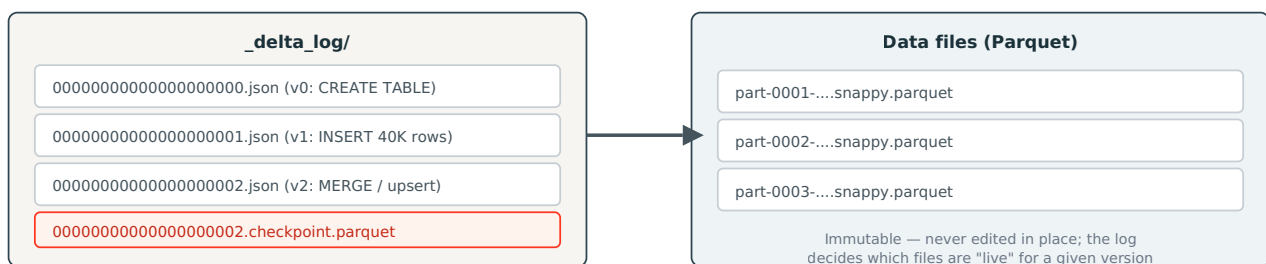


Figure 4.1 — A Delta table is a directory of immutable Parquet files plus a transaction log. Each commit adds a new log entry; readers reconstruct the correct file set for any version from the log.

4.2 The Medallion Architecture

Databricks recommends organizing Delta tables into three progressively refined layers, commonly called the **medallion architecture**:

- **Bronze** — raw, largely unmodified ingestion from source systems, kept for auditability and reprocessing.
- **Silver** — cleaned, deduplicated, conformed data with enforced schemas, typically joined across sources.
- **Gold** — business-level aggregates and marts, optimized for BI tools and reporting.

EXAMPLE — PYTHON / PYSPARK: BRONZE-TO-SILVER TRANSFORMATION

```
from pyspark.sql import functions as F

# Read raw bronze events (append-only ingestion table)
bronze_df = spark.readStream.table("main.bronze.raw_orders")

silver_df = (
    bronze_df
    .dropDuplicates(["order_id"])
    .filter(F.col("order_status").isNotNull())
    .withColumn("order_ts", F.to_timestamp("order_ts"))
    .withColumn("ingest_date", F.current_date())
)

(
    silver_df.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", "/checkpoints/silver_orders")
    .trigger(availableNow=True)
)
```

```
.toTable("main.silver.orders_clean")  
)
```

4.3 Performance: Liquid Clustering, Z-Ordering, and Predictive I/O

Delta Lake tables are optimized for query performance using **Z-Ordering** (co-locating related data within files for a fixed set of columns) and its successor, **Liquid Clustering**, which incrementally reclusters data without full-table rewrites and adapts as query patterns change. **Predictive I/O**, layered on top, uses machine-learned models over table statistics to prune files and rows before they are ever scanned, further reducing the data volume read for a given query.

RULE OF THUMB

Use liquid clustering on new tables instead of partitioning by a low-cardinality column; it avoids the classic "too many small partitions" problem and lets you change the clustering keys later without rewriting the table from scratch.

SECTION 5

Compute Architecture: Clusters, SQL Warehouses, and Photon

The processing layer that reads and writes Delta tables — sized and billed independently of storage.

5.1 All-Purpose and Job Clusters

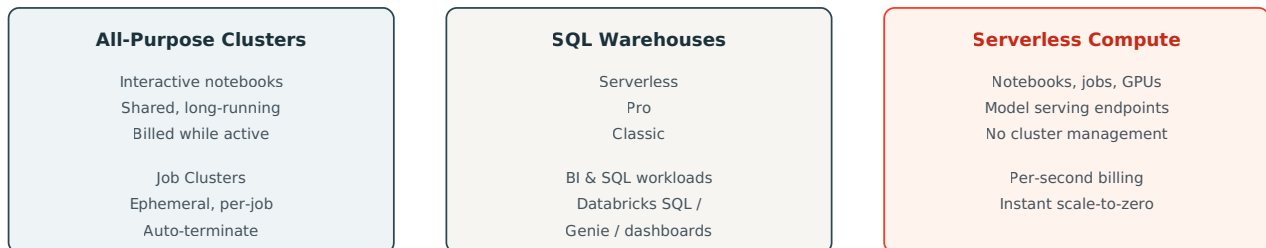
Clusters are groups of VMs running Apache Spark. **All-purpose clusters** are long-lived, shared by multiple users for interactive notebook work, and billed while running regardless of utilization. **Job clusters** are created automatically for a scheduled job, run only for the duration of that job, and are terminated immediately afterward — typically the cheaper option for production pipelines since there is no idle time.

5.2 SQL Warehouses: Serverless, Pro, and Classic

SQL Warehouses are purpose-built compute for SQL and BI workloads, exposed through Databricks SQL. Three variants trade off control, startup latency, and feature access:

Warehouse Type	Startup Time	Predictive I/O	Notes
Serverless	Seconds	Yes	Fully managed by Databricks; auto-scales; supports intelligent workload management
Pro	~2–3 minutes	Yes	Customer cloud account; supports Photon and predictive I/O, not intelligent workload management
Classic	~4 minutes	No	Customer cloud account; Photon enabled, least responsive autoscaling

Databricks Compute Family



All compute types execute against the same underlying Delta tables, accelerated by the Photon engine

Figure 5.1 — Compute is chosen per workload: interactive clusters for notebooks, SQL warehouses for BI/SQL, serverless for elastic, low-ops execution.

5.3 The Photon Engine

Photon is a native, vectorized query engine (written in C++, not JVM-based) that replaces parts of the Spark execution engine for SQL and DataFrame operations. It is enabled by default on serverless compute, SQL warehouses, and classic all-purpose/job clusters, and gives the largest gains on scan-, join-, and aggregation-heavy workloads. Photon works transparently — existing Spark SQL and DataFrame code does not need to change to benefit from it.

EXAMPLE — PYTHON: CHECKING AND FORCING PHOTON ON A CLUSTER-ATTACHED SESSION

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()

# Confirm Photon is active for this session
photon_enabled = spark.conf.get("spark.databricks.photon.enabled", "unknown")
print(f"Photon enabled: {photon_enabled}")

# A typical aggregation that benefits from Photon's vectorized execution
```

```
result = (  
    spark.table("main.gold.daily_sales")  
    .groupBy("region", "product_category")  
    .sum("revenue")  
    .orderBy("sum(revenue)", ascending=False)  
)  
result.show(10)
```

SECTION 6

Unity Catalog: Unified Governance for Data and AI

The single governance plane that spans tables, files, models, notebooks, dashboards, and — as of 2026 — AI agents.

Unity Catalog is Databricks' centralized governance layer. Before its introduction, permissions, lineage, and auditing were managed per-workspace and per-cluster, which meant governance did not travel with the data across workspaces or clouds. Unity Catalog fixes this by attaching governance metadata to the data itself, at the account level, so a single set of policies is enforced consistently across AWS, Azure, and GCP workspaces.

6.1 The Three-Level Namespace and Metastore

Every governed asset is addressed with a three-level namespace: `catalog.schema.table`. A **metastore** is the top-level container for metadata, typically one per region, attached to every workspace that should share the same governance policies.

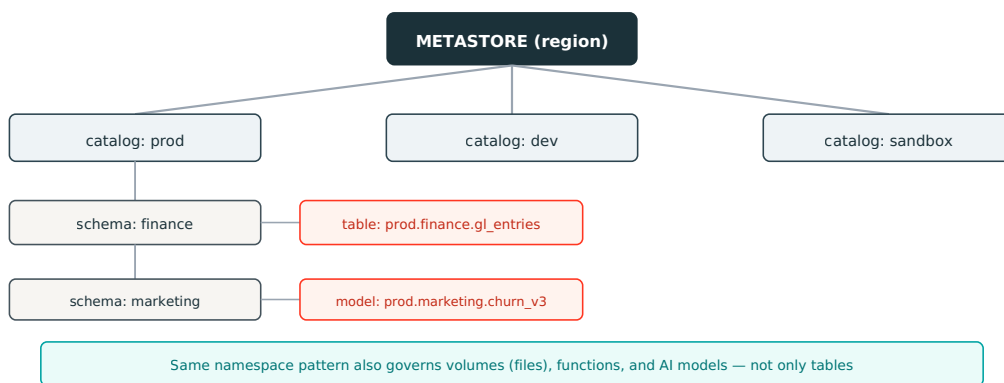


Figure 6.1 — Unity Catalog's three-level namespace: metastore > catalog > schema > securable object (table, view, volume, model, or function).

EXAMPLE — SQL: CREATING A GOVERNED TABLE AND GRANTING ACCESS

```
-- Create a catalog and schema (one-time setup, typically by a platform admin)
CREATE CATALOG IF NOT EXISTS prod;
CREATE SCHEMA IF NOT EXISTS prod.finance;

-- Create a managed Delta table inside Unity Catalog
CREATE TABLE prod.finance.gl_entries (
  entry_id BIGINT,
  account_code STRING,
  amount DECIMAL(18,2),
  posted_date DATE
) USING DELTA;

-- Grant read access to an analyst group, write access to the ETL service principal
GRANT SELECT ON TABLE prod.finance.gl_entries TO `finance-analysts`;
GRANT MODIFY ON TABLE prod.finance.gl_entries TO `svc-finance-etl`;
```

6.2 Governance Object Model

Unity Catalog governs more than tables. Its object model includes **volumes** (governed access to non-tabular files, such as PDFs or images — directly relevant to a document-upload RAG pipeline), **functions** (registered SQL/Python UDFs), **models** (registered ML/AI models), and **external locations/storage credentials** (governed pointers to raw cloud storage paths outside of managed tables). Every object type shares the same `GRANT / REVOKE` permission syntax and the same audit log.

RELEVANT TO A DOCUMENT RAG PIPELINE

A Unity Catalog **Volume** is the natural place to land uploaded PDFs before chunking: `/Volumes/prod/rag/uploaded_docs/` gives you governed, access-controlled file storage with the same permission model as your vector index tables, so a single Unity Catalog policy can restrict who is allowed to upload, query, or view source documents.

6.3 2026 Additions: Business Glossary, Domains, and Governance Hub

Announced at the 2026 Data + AI Summit, Unity Catalog extended from a purely technical metadata layer into a semantic one. **Business Glossary** lets organizations define terms (e.g., "active customer," "churn") once, centrally. **Domains** group related data and AI assets by business unit or subject area rather than by workspace. The **Governance Hub** extends policy enforcement beyond stored data to the runtime interactions between models, agents, and tools — governing not just "who can read this table" but "what is this AI agent allowed to do with it." A related capability, **Metrics**, allows a KPI (revenue, churn, active users) to be defined once as a governed, reusable object and queried consistently from SQL, BI tools, APIs, and AI agents.

SECTION 7

Data Governance Models: Access Control, Lineage, and Compliance

How Unity Catalog implements access control models in practice, down to the row and column.

7.1 RBAC and ABAC Access Models

Databricks supports both classical access control patterns, and most enterprise deployments use a blend of the two:

- **Role-Based Access Control (RBAC).** Permissions are granted to groups (roles) rather than individual users — e.g., `finance-analysts` gets `SELECT` on the finance schema. This is simple to audit and is the default pattern for most `GRANT` statements.
- **Attribute-Based Access Control (ABAC).** Access decisions incorporate attributes of the user, the data, or the context (e.g., "mask this column unless the requester's department attribute equals Finance and it is a business day"). Unity Catalog implements ABAC-style policies through tag-based masking policies and row filters that evaluate dynamically at query time.

Query-time access-control flow

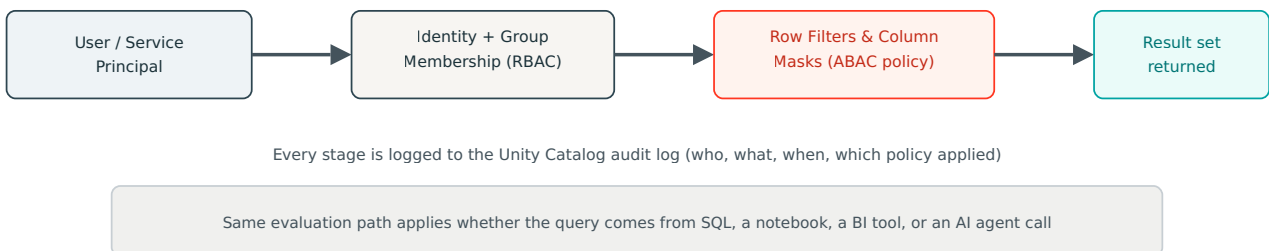


Figure 7.1 — Access evaluation combines group-based grants (RBAC) with dynamic row/column policies (ABAC) at query time, before any data is returned.

7.2 Row, Column, and Dynamic View Security

Fine-grained security in Unity Catalog is implemented three ways: **row filters** (a function that determines which rows a given caller may see), **column masks** (a function that redacts or transforms a column's value based on caller identity), and **dynamic views** (a view definition that references `current_user()` or `is_account_group_member()` directly in its SQL).

EXAMPLE — SQL: ROW FILTER AND COLUMN MASK

```

-- Column mask: only HR sees full salary, everyone else sees NULL
CREATE FUNCTION prod.hr.mask_salary(salary DECIMAL(10,2))
RETURN CASE
  WHEN is_account_group_member('hr-team') THEN salary
  ELSE NULL
END;

ALTER TABLE prod.hr.employees
  ALTER COLUMN salary SET MASK prod.hr.mask_salary;

-- Row filter: regional managers only see rows for their own region
CREATE FUNCTION prod.sales.region_filter(region STRING)
RETURN region = current_user_region() OR is_account_group_member('global-admins');

ALTER TABLE prod.sales.regional_orders
  SET ROW FILTER prod.sales.region_filter ON (region);
  
```

7.3 Lineage, Audit, and Classification

Unity Catalog automatically captures **column-level lineage** across notebooks, SQL, Workflows, and Lakeflow pipelines, showing exactly which upstream tables and columns fed a downstream table or dashboard — essential for impact analysis and regulatory data lineage requirements (e.g., BCBS 239, GDPR data mapping). Every read and write is captured in the **audit log**, which can be exported to SIEM tooling. **Tags** (e.g., `pii=true`, `classification=confidential`) can be applied to catalogs, schemas, tables, or columns, and combined with attribute-based masking policies to enforce classification-driven access at scale rather than one grant at a time.

Capability	Governs	Typical Compliance Use
Lineage graph	Table/column-level upstream-downstream relationships	Impact analysis, regulatory data mapping
Audit log	Every access event, per principal, per object	SOC 2 / HIPAA access reviews, SIEM export
Tags & classification	Sensitivity labeling at column/table level	PII/PHI discovery, policy-driven masking
Row filters / column masks	Data returned, per query, per caller	Multi-tenant isolation, need-to-know access

SECTION 8

Data Sharing and Interoperability

Moving data across organizational and platform boundaries without copying it, or querying it in place without moving it at all.

8.1 Delta Sharing

Delta Sharing is an open protocol for sharing live data with any computing platform — the recipient does not need to run Databricks. It works over standard REST semantics with short-lived, pre-signed URLs for the underlying files, end-to-end TLS encryption, and no need to copy or replicate the shared dataset. Because it is integrated with Unity Catalog, every share is centrally governed, permissioned, and appears in the audit log exactly like an internal grant. Since 2022, recipients can also authenticate through their own identity provider (e.g., Microsoft Entra ID, Google identity) rather than exchanging static bearer tokens.

EXAMPLE — SQL: CREATING AND GRANTING A DELTA SHARE

```
CREATE SHARE partner_sales_share
  COMMENT 'Quarterly aggregated sales data for retail partner';

ALTER SHARE partner_sales_share
  ADD TABLE prod.gold.quarterly_sales_by_region;

CREATE RECIPIENT retail_partner_co
  USING ID 'retail-partner-sharing-identifier';

GRANT SELECT ON SHARE partner_sales_share TO RECIPIENT retail_partner_co;
```

8.2 Lakehouse Federation

Lakehouse Federation is the inverse problem: querying data that lives in an external system — Snowflake, BigQuery, PostgreSQL, MySQL, Redshift — without first migrating it into Delta Lake. Unity Catalog registers a **foreign catalog** pointing at the external system's connection details, and the same grant/audit/lineage machinery applies to federated queries as to native Delta tables. This is typically used to avoid a full migration up front, or to join governed lakehouse data against a system of record that must legally stay in place.

SHARING VS. FEDERATION — WHICH ONE?

Use **Delta Sharing** when you want to expose your own data to external consumers with minimal integration effort. Use **Lakehouse Federation** when you need to pull in and join against someone else's live system without waiting for it to be migrated or replicated.

SECTION 9

Orchestration and Pipelines: Lakeflow and Workflows

Scheduling, dependency management, and declarative ETL for moving data through bronze, silver, and gold.

9.1 Databricks Workflows and Jobs

Workflows is the native orchestrator: a job is a DAG of tasks (notebooks, SQL files, Python scripts, dbt models, or pipelines), each with its own compute, retry policy, and dependency conditions. Jobs can be triggered on a schedule, by a file arrival, or via API/webhook, and support task-level parameterization so the same job definition can run against different environments (dev/staging/prod catalogs).

9.2 Lakeflow: Connect, Declarative Pipelines, and Jobs

Lakeflow is Databricks' unified data engineering product line, encompassing three components: **Lakeflow Connect** (managed, low-code ingestion connectors for SaaS applications and databases), **Lakeflow Declarative Pipelines** (formerly Delta Live Tables — a declarative framework where you define the desired end-state of a table and Databricks manages the incremental execution, checkpointing, and orchestration), and **Lakeflow Jobs** (the underlying scheduling engine, shared with Workflows).

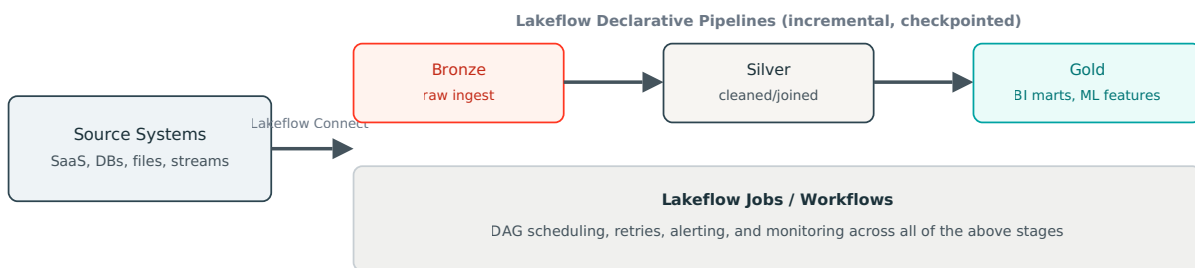


Figure 9.1 — Lakeflow Connect ingests raw data; Declarative Pipelines move it through bronze/silver/gold; Lakeflow Jobs (Workflows) schedule and monitor the whole DAG.

EXAMPLE — PYTHON: A LAKEFLOW DECLARATIVE PIPELINE DEFINITION

```

import dlt
from pyspark.sql import functions as F

@dlt.table(
    comment="Raw customer support tickets, ingested as-is",
    table_properties={"quality": "bronze"}
)
def bronze_tickets():
    return spark.readStream.format("cloudFiles") \
        .option("cloudFiles.format", "json") \
        .load("/Volumes/prod/raw/support_tickets/")

@dlt.table(
    comment="Cleaned, deduplicated support tickets",
    table_properties={"quality": "silver"}
)
@dlt.expect_or_drop("valid_ticket_id", "ticket_id IS NOT NULL")
def silver_tickets():
    return (
        dlt.read_stream("bronze_tickets")
        .dropDuplicates(["ticket_id"])
        .withColumn("resolved_ts", F.to_timestamp("resolved_at"))
    )
  
```


SECTION 10

AI and Machine Learning: MLflow, Mosaic AI, and Genie

Model training, generative AI, and conversational analytics — all built on top of the same governed Delta/Unity Catalog layer.

10.1 MLflow and the Model Lifecycle

MLflow, originally created by Databricks and now a widely adopted open-source standard, manages the ML lifecycle across four components: **Tracking** (logging parameters, metrics, and artifacts for each training run), the **Model Registry** (versioning models and promoting them through stages such as Staging and Production), **Projects** (packaging reproducible training code), and **Model Serving** (deploying a registered model behind a low-latency REST endpoint). Registered models live inside Unity Catalog alongside tables, so a model's access grants and lineage back to its training data are managed the same way.

EXAMPLE — PYTHON: LOGGING AND REGISTERING A MODEL WITH MLFLOW

```
import mlflow
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

mlflow.set_registry_uri("databricks-uc")

with mlflow.start_run(run_name="churn_model_v3") as run:
    model = RandomForestClassifier(n_estimators=200, max_depth=8)
    model.fit(X_train, y_train)

    preds = model.predict(X_test)
    acc = accuracy_score(y_test, preds)

    mlflow.log_param("n_estimators", 200)
    mlflow.log_param("max_depth", 8)
    mlflow.log_metric("accuracy", acc)

    mlflow.sklearn.log_model(
        model,
        artifact_path="model",
        registered_model_name="prod.marketing.churn_model"
    )
```

10.2 Mosaic AI: Governed Generative AI

Mosaic AI is Databricks' layer for building and operating generative AI applications directly against governed data. It exposes a single API across multiple model providers (open-source and proprietary LLMs), enabling routing, usage monitoring, safety guardrails, and cost controls without changing application code. **Agent Bricks** provides templated patterns for building and evaluating autonomous or semi-autonomous agents, and **Vector Search** is the managed component most relevant to RAG applications: it indexes embeddings stored in a Delta table and keeps the index continuously synced as the underlying table changes, so retrieval always reflects the current governed data.

DIRECT RELEVANCE TO YOUR RAG PROJECT

Databricks Vector Search is a managed analogue to the hybrid-search + re-ranking pipeline you are building: it supports both dense (embedding) and sparse (keyword/BM25) retrieval in a single index, callable from a Model Serving endpoint. Even if your FastAPI service uses its own vector store, the underlying design pattern — governed source table, embedding sync job, hybrid query, then an LLM call over the retrieved chunks — mirrors Mosaic AI's own architecture.

10.3 Genie: Conversational Analytics

Genie is Databricks' natural-language interface for business analytics: a user asks a question in plain English, and Genie generates and runs the underlying SQL against governed Unity Catalog tables, returning both an answer and the query used to produce it. The 2026 rebuild added **Genie Ontology** — a semantic context layer that continuously reads tables, documents, tickets, and connected apps to build a live map of business meaning — and **Genie One**, which unifies conversational analytics with agentic task execution in a single interface.

SECTION 11

Security, Compliance, and Enterprise Readiness

The network, identity, and compliance controls that sit around the architecture and governance layers already described.

Security on Databricks is layered across network, identity, and encryption controls, in addition to the Unity Catalog governance model already covered in Sections 6 and 7:

- **Network isolation.** Customer-managed VPC/VNet injection, private connectivity (AWS PrivateLink, Azure Private Link) between the control plane and data plane, and IP access lists restricting workspace access to known corporate ranges.
- **Identity.** SSO/SCIM integration with an enterprise identity provider, service principals for automated workloads (jobs, pipelines) instead of personal credentials, and OAuth machine-to-machine authentication for API access.
- **Encryption.** Data encrypted at rest (cloud-provider-managed or customer-managed keys) and in transit (TLS) by default, with customer-managed key (CMK) support for organizations with key-rotation or key-custody requirements.
- **Compliance.** Databricks maintains certifications and attestations widely relied upon by regulated industries, including SOC 2 Type II, ISO 27001, HIPAA (with a signed BAA), PCI DSS, and FedRAMP Moderate/High for applicable government deployments. Specific certifications vary by cloud and region, so they should be confirmed against current Databricks compliance documentation for any specific deployment.

CAVEAT FOR REGULATED ENVIRONMENTS

Compliance certifications and supported regions change over time and differ across AWS, Azure, and GCP deployments of Databricks. Treat the list above as a starting checklist, not a substitute for verifying current certification status directly with Databricks for your specific cloud, region, and workspace tier.

SECTION 12

Best Practices, Roadmap & Conclusion

A consolidated checklist that pulls together the architecture and governance material into an adoption sequence.

Stage	Focus	Key Actions
1	Foundational architecture	Stand up a metastore per region; decide classic vs. serverless data plane per workload; configure private connectivity between control and data planes.
2	Storage & layering	Adopt the medallion architecture (bronze/silver/gold); default new tables to liquid clustering rather than static partitioning.
3	Governance baseline	Migrate all catalogs into Unity Catalog; define a tagging taxonomy for PII/PHI before data volume grows; set up row filters/column masks for regulated tables.
4	Compute rightsizing	Use job clusters (not all-purpose) for production pipelines; move ad hoc BI to serverless SQL warehouses to cut idle spend.
5	Orchestration	Consolidate ingestion on Lakeflow Connect; move hand-rolled ETL notebooks to declarative pipelines where feasible.
6	AI enablement	Register models and vector indexes in Unity Catalog so governance and lineage extend to AI artifacts, not just tables.

COMMON PITFALL

Treating Unity Catalog migration as a "later" project after pipelines are already in production is the single most common source of retrofitting pain — permissions, lineage, and tagging are far cheaper to establish before table count and workspace count grow.

12.1 Conclusion

The Databricks platform is best understood as two intertwined systems: an **architecture** that separates ephemeral, elastic compute (clusters, warehouses, serverless) from durable, transactional storage (Delta Lake on open object storage), and a **governance layer** (Unity Catalog) that follows the data — not the compute — across workspaces, clouds, and now AI agents. Every capability covered in this document, from medallion pipelines to Genie's conversational analytics, is built as a consumer of that same governed storage layer, which is what allows a single access grant to protect a table whether it is queried by SQL, a notebook, a BI dashboard, or an LLM-based agent.

For a Databricks professional, the practical takeaway is that architecture decisions (which compute type, which plane) and governance decisions (which catalog, which masking policy) should be made together, not sequentially — retrofitting governance onto an already-sprawling set of ungoverned tables is markedly more expensive than establishing it from the first catalog onward.

SECTION 13

References

Primary sources consulted in compiling this reference. Product names, features, and version details should be re-verified against current Databricks documentation before being relied on operationally, as the platform evolves frequently.

-
- [1] **Databricks, Inc. — "Databricks reference architectures."** Databricks Documentation (AWS/Azure/GCP). <https://docs.databricks.com/aws/en/lakehouse-architecture/reference>

 - [2] **Databricks, Inc. — "What is Photon?"** Databricks Documentation. <https://docs.databricks.com/aws/en/compute/photon>

 - [3] **Databricks, Inc. — "SQL warehouse types."** Databricks Documentation. <https://docs.databricks.com/aws/en/compute/sql-warehouse/warehouse-types>

 - [4] **Databricks, Inc. — "Best practices for interoperability and usability."** Databricks Documentation. <https://docs.databricks.com/aws/en/lakehouse-architecture/interoperability-and-usability/best-practices>

 - [5] **Databricks, Inc. — "What's new with Unity Catalog at Data + AI Summit 2026."** Databricks Blog. <https://www.databricks.com/blog/whats-new-unity-catalog-data-ai-summit-2026>

 - [6] **Databricks, Inc. — "What's new with Data Sharing & Collaboration."** Databricks Blog. <https://www.databricks.com/blog/whats-new-data-sharing-collaboration>

 - [7] **Databricks Community — "Delta Sharing: Open, Secure, and Barrier-Free Data & AI."** Databricks Community Data Governance forum. <https://community.databricks.com/t5/data-governance/delta-sharing-open-secure-and-barrier-free-data-amp-ai/td-p/135371>

 - [8] **Databricks, Inc. — "Security best practices for Delta Sharing."** Databricks Blog. <https://databricks.com/blog/2022/08/01/security-best-practices-for-delta-sharing.html>

 - [9] **Atlan — "Databricks Data + AI Summit 2026: Key Announcements Recap."** Atlan Knowledge Base. <https://atlan.com/know/ai-agent/databricks/databricks-data-ai-summit-2026-announcements/>

 - [10] **Qubika — "Everything Databricks Announced at the DAIS Data + AI Summit 2026."** Qubika Blog. <https://qubika.com/blog/everything-databricks-announced-dais-data-ai-summit-2026/>

 - [11] **Datapao — "Databricks Data + AI Summit 2026: Key Announcements Explained."** Datapao Blog. <https://datapao.com/databricks-data-ai-summit-2026/>

 - [12] **Flexera — "Databricks Unity Catalog 101: A Complete Overview (2026)."** Flexera FinOps Blog. <https://www.flexera.com/blog/finops/databricks-unity-catalog/>

 - [13] **Flexera — "Databricks SQL Warehouse: Serverless vs Pro vs Classic (2026)."** Flexera FinOps Blog. <https://www.flexera.com/blog/finops/databricks-sql-warehouse-types/>

 - [14] **lakeFS — "Databricks Architecture Overview: Components & Key Features."** lakeFS Blog. <https://lakefs.io/blog/databricks-architecture/>

 - [15] **Armbrust, M. et al. — "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics."** Proceedings of CIDR 2021.

 - [16] **Databricks, Inc. — MLflow Documentation.** <https://mlflow.org/docs/latest/index.html>

 - [17] **Databricks, Inc. — Delta Lake Documentation.** <https://docs.delta.io/latest/index.html>

DOCUMENT METADATA

Generated July 26, 2026, 20:21 PDT, for use as a source document in a retrieval-augmented generation (RAG) pipeline test corpus. Content reflects publicly available Databricks documentation and 2026 product announcements at time of writing; verify against current docs.databricks.com before operational use.